
A6\code\A6.py

```
## Hier werden saemtliche Pakete und Module geladen, die fuer das Notebook benoetigt sind.
from utils import *
import numpy as np
# BEGIN SOLUTION
import scipy.sparse
# END SOLUTION
np.random.seed(0)

# TODO: Aufgabenteil 1: Implementiere Standardform des CG-Verfahrens
def conjugate_gradient(A, b, c, x0, tol=1e-10, kmax=100, xstar=None, plot=False):
    """
    Standardform des CG-Verfahrens

    Input:
        A: Matrix (spd!),
        b: Vektor,
        c: Skalar,
        x0: Vektor (Startwert),
        tol: Skalar (Toleranz),
        kmax: Skalar (Maximale Anzahl an Iterationen),
        xstar: Vektor (Minimierer von f), optional.
        plot: Boolescher Wert (gibt an, ob Daten zur Ploterstellung gespeichert werden sollen).

    Output:
        log: Dictionary vom Verlauf der Optimierung.
    """
    # Quadratische Funktion, die (val, grad, hess) der zugehörigen quadratischen Funktion zurückgibt.
    f = lambda x: quadratic_function(x, A, b, c)

    # Dictionary zum Abspeichern der Ergebnisse.
    log = {
        'xstar': xstar,          # Tatsächlicher Minimierer
        'val_star': None if xstar is None else f(xstar)[0], # Tatsächliches Minimum
        'x0': x0,               # Startwert
        'xcg': None,            # Lösung des CG-Verfahrens
        'kcg': None,            # Anzahl benötigter Iterationen
        'x_list': [],           # Liste der Iterierten
        'val_list': [],         # Liste des Funktionswerts in den Iterierten
    }
```

```

    'norm_grad_list': [],      # Liste der Norm des Gradienten in den Iterierten
}

# BEGIN SOLUTION

#initialisierung
xk = x0
rk = b - A @ xk
pk = rk

#initiales logging
log['x_list'].append(xk.copy())
if plot:
    val, grad, _ = f(xk)
    log['val_list'].append(val)
    log['norm_grad_list'].append(np.linalg.norm(grad))

# iterative Berechnung
for k in range(1,kmax+1):
    test_konjugent = pk @ A # erster Teil vom Test
    vk = A @ pk
    alphak = rk @ rk / np.inner(pk,vk)
    xk = xk + alphak * pk

    #logging
    log['x_list'].append(xk.copy())
    if plot:
        val, grad, _ = f(xk)
        log['val_list'].append(val)
        log['norm_grad_list'].append(np.linalg.norm(grad))

    betak = 1 / (rk @ rk) # erster Teil von beta
    rk = rk - alphak * vk
    if np.linalg.norm(rk) <= tol:
        log['xcg'] = xk.copy()
        log['kcg'] = k
        break
    betak = rk @ rk * betak # zweiter Teil von beta
    pk = rk + betak * pk
    test_konjugent = test_konjugent @ pk # zweiter Teil vom Test
    if not (test_konjugent < 1e-10):
        print("Im Schritt ",k," sind die aufeinanderfolgenden Richtungen nicht konjugiert!\n")

else:
    log['xcg'] = xk.copy()
    log['kcg'] = kmax
# END SOLUTION
return log

```

```

def test_aufgabenteil_2():
    # TODO: Aufgabenteil 2: Teste Implementierung auf 2-dim. Funktionen.
    # Hinweis: Die plot-Funktion aus utils.py soll nur die Plots erstellen und die Figure zur  ckgeben.
    # Stellen Sie die Abbildung bitte dar, indem Sie HIER plt.show() aufrufen.

    kmax = 200

```

```

tol = 1e-10
b = np.array([3, 6])
c = np.pi
x0 = np.array([0, 0])

A1 = np.array([[13, 0], [0, 13]])
A2 = np.array([[1, 1 / 2], [1 / 2, 1]])
A3 = np.array([[100, 0], [0, 1]])

# BEGIN SOLUTION
matrizen = [A1,A2,A3]
for i in range(3):
    f = lambda x: quadratic_function(x, matrizen[i], b, c)
    xstar = np.linalg.solve(matrizen[i], b)
    log_cg = conjugate_gradient(matrizen[i], b, c, x0, tol=1e-10, kmax=100, xstar=xstar, plot=True)
    matrixname = ''.join(["A",str(i+1)])
    figure_cg = plot_iteration_process(log_cg, "CG: " + matrixname, f)
    plt.show()

    log_ga = gradient_descent(f, x0, tol, kmax, xstar)
    figure_gd = plot_iteration_process(log_ga, "Gradientenabstieg: " + matrixname, f)
    plt.show()
print("Das CG-Verfahren benötigt jeweils nur 2 Schritte.\n")
    "Das Gradientenabstiegsverfahren wiederum benötigt bei der A2 und A3 Matrix deutlich mehr Itera
    "Bei der letzten wird sogar das kmax-Abbruchskriterium erreicht.\n")
# END SOLUTION
return

def test_aufgabenteil_3():
    # TODO: Aufgabenteil 3: Teste Implementierung auf Tridiagonalmatrix.
    # Hinweis: Importieren Sie das benötigte Paket am Anfang der Python-Datei wie folgt: import scipy.sparse

    print('-----')
    print('Test mit A4')
    print('-----')
    # BEGIN SOLUTION
    # Matrix A und Vektor b:
    A4 = scipy.sparse.spdiags(np.ones((1, 250)) * np.array([[ -1], [2], [ -1]]),
                              np.array([ -1, 0, 1]), 250, 250).toarray()
    b4 = 0.1 * np.random.randn(250)

    #sonstige Initialisierungen:
    tol = 1e-10
    kmax = 3000
    xstar = np.linalg.solve(A4, b4)
    x0 = np.zeros(250)
    c = 0
    f = lambda x: quadratic_function(x, A4, b4, c)

    # Berechnungen und Plots
    # Gradientenabstiegsverfahren:
    log_ga = gradient_descent(f, x0, tol, kmax, xstar)
    figure_ga = plot_iteration_process(log_ga, "Gradientenabstieg: A4", f=None)
    print("\nGradientenabstiegsverfahren:")
    print("Schritte: ", log_ga['kgd'], "\n")

```

```

print("Residuumsnorm: ", np.linalg.norm(A4 @ log_ga['xgd'] - b4), "\n")
print("Normdifferenz zu xstar: ", np.linalg.norm(log_ga['xgd'] - xstar), "\n")
plt.show()

# CG-Verfahren:
log_cg = conjugate_gradient(A4, b4, c, x0, tol, kmax, xstar, plot=True)
figure_cg = plot_iteration_process(log_cg, "CG: A4", f=None)
print("\nCG-Verfahren:")
print("Schritte: ", log_cg['kcg'], "\n")
print("Residuumsnorm: ", np.linalg.norm(A4 @ log_cg["xcg"] - b4), "\n")
print("Normdifferenz zu xstar: ", np.linalg.norm(log_cg["xcg"] - xstar), "\n")
plt.show()
# END SOLUTION
return

```

```

def test_aufgabenteil_4():
    # TODO: Aufgabenteil 4.i: Teste Implementierung auf Normalgleichung mit A5.
    print('\n-----')
    print('Test mit A5')
    print('-----')
    # BEGIN SOLUTION
    # Initialisierungen
    A5 = np.array([[1, 2], [3, 4], [5, 6]])
    b5 = np.array([np.pi, 2, 1])
    tol = 1e-2
    c = 0
    x0 = np.zeros(2)

    # Matrizenrechnungen
    ATA = A5.T @ A5
    ATb = A5.T @ b5

    f = lambda x: quadratic_function(x, ATA, ATb, c)
    log_cg = conjugate_gradient(ATA, ATb, c, x0)
    min_norm_lsg = log_cg['xcg']
    xstar = np.linalg.solve(ATA, ATb)
    print("Die Differenz zwischen der berechneten Minimum-Norm-Lösung"
          "und dem Ergebnis von np.linalg.solve() für A5 und b5 ist: ",
          np.linalg.norm(min_norm_lsg-xstar), "\n")
    # END SOLUTION

    # TODO: Aufgabenteil 4.ii: Teste Implementierung auf Normalgleichung mit A6.
    print('\n-----')
    print('Test mit A6')
    print('-----')
    # BEGIN SOLUTION
    A6 = np.diag(np.arange(1, 101))
    b6 = np.random.rand(100)
    x0 = np.zeros(100)

    f1 = lambda x: quadratic_function(x, A6, b6, c)
    xstar1 = np.linalg.solve(A6, b6)
    log1 = conjugate_gradient(A6, b6, c, x0, tol=tol)
    print("Benötigte Iterationen: ", log1["kcg"])

```

```

# END SOLUTION

# TODO: Aufgabenteil 4.ii: Teste Implementierung auf Normalgleichung mit A6mod.
print('\n-----')
print('test mit A6mod')
print('-----')
# BEGIN SOLUTION
A6mod = A6.T @ A6
b6mod = A6.T @ b6

f2 = lambda x: quadratic_function(x, A6mod, b6mod, c)
xstar2 = np.linalg.solve(A6mod, b6mod)
log2 = conjugate_gradient(A6mod, b6mod, c, x0, tol=tol)
print("Benötigte Iterationen: ", log2["kcg"])

print("Die modifizierte Matrix/Vektor benötigen deutlich mehr Iterationen.\n"
      "Außerdem sind bei dieser für viele Suchrichtungen jeweils 2 aufeinanderfolgende nicht konjugiert.")
# END SOLUTION
return

if __name__ == '__main__':
    test_aufgabenteil_2()
    test_aufgabenteil_3()
    test_aufgabenteil_4()

```

#TERMINAL OUTPUT:

Das CG-Verfahren benötigt jeweils nur 2 Schritte.

Das Gradientenabstiegsverfahren wiederum benötigt bei der A2 und A3 Matrix deutlich mehr Iterationen.

Bei der letzten wird sogar das kmax-Abbruchskriterium erreicht.

Test mit A4

Gradientenabstiegsverfahren:

Schritte: 3000

Residuumsnorm: 0.08234395753463068

Normdifferenz zu xstar: 324.63007424657667

CG-Verfahren:

Schritte: 250

Residuumsnorm: 5.966007726004174e-13

Normdifferenz zu xstar: 8.643233334719968e-12

Test mit A5

Die Differenz zwischen der berechneten Minimum-Norm-Lösung und dem Ergebnis von `np.linalg.solve()` für A5 und b5 ist: 1.1580429779916289e-14

Test mit A6

Benötigte Iterationen: 27

test mit A6mod

Im Schritt 1 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 2 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 3 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 4 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 5 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 6 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 7 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 8 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 9 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 10 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 11 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 12 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 13 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 14 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 15 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 16 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 17 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 18 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 19 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 20 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 21 sind die aufeinanderfolgenden Richtungen nicht konjugiert!
Im Schritt 22 sind die aufeinanderfolgenden Richtungen nicht konjugiert!

Im Schritt 23 sind die aufeinanderfolgenden Richtungen nicht konjugiert!

Im Schritt 24 sind die aufeinanderfolgenden Richtungen nicht konjugiert!

Im Schritt 25 sind die aufeinanderfolgenden Richtungen nicht konjugiert!

Im Schritt 26 sind die aufeinanderfolgenden Richtungen nicht konjugiert!

Im Schritt 27 sind die aufeinanderfolgenden Richtungen nicht konjugiert!

Im Schritt 28 sind die aufeinanderfolgenden Richtungen nicht konjugiert!

Im Schritt 29 sind die aufeinanderfolgenden Richtungen nicht konjugiert!

Im Schritt 30 sind die aufeinanderfolgenden Richtungen nicht konjugiert!

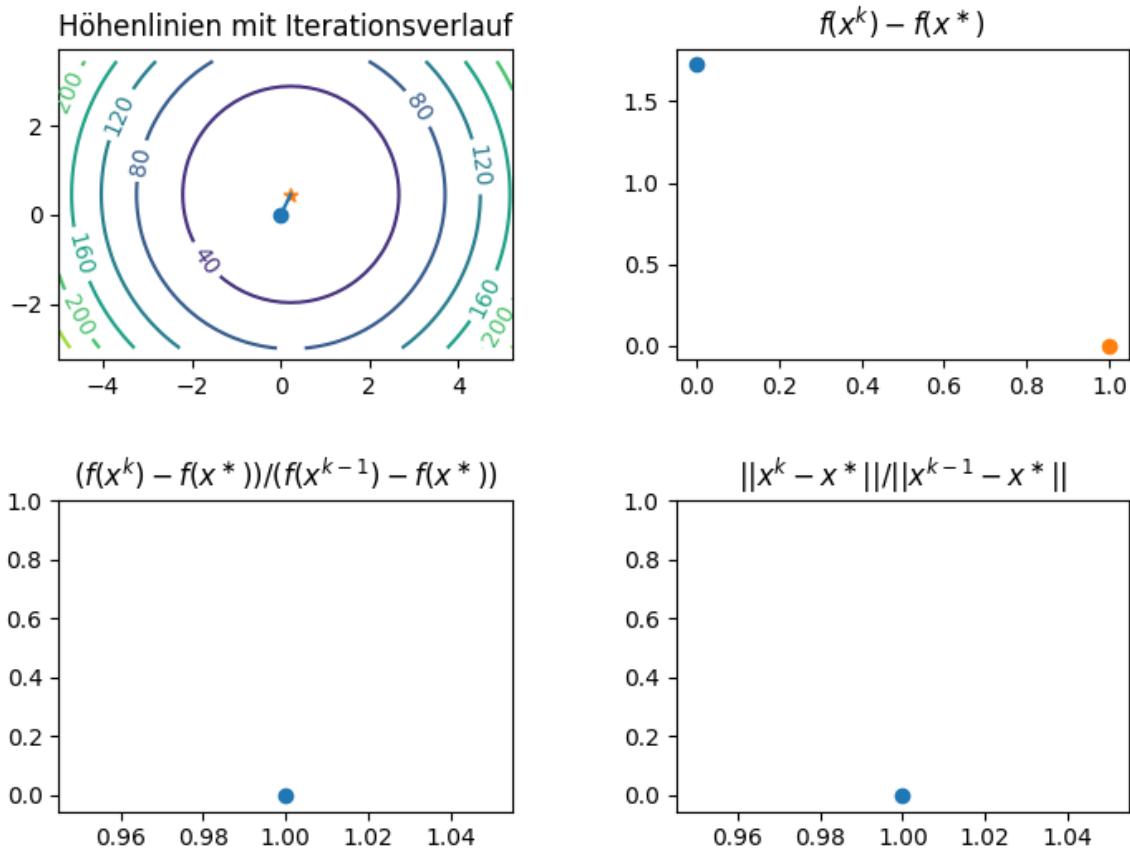
Benötigte Iterationen: 100

Die modifizierte Matrix/Vektor benötigen deutlich mehr Iterationen.

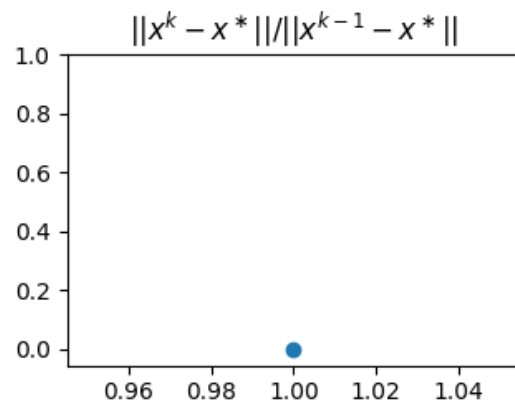
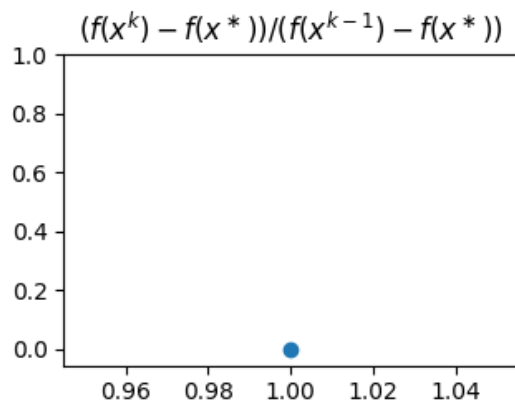
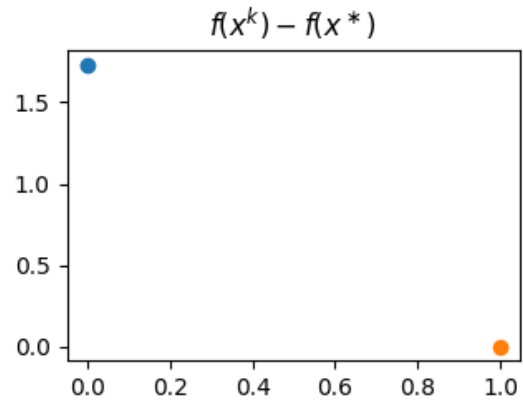
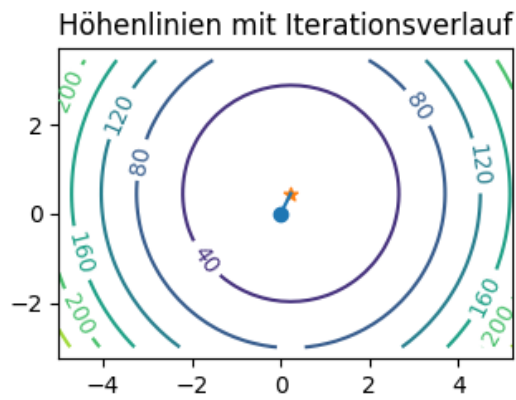
Außerdem sind bei dieser für viele Suchrichtungen jeweils 2 aufeinanderfolgende nicht konjugiert zueinander.

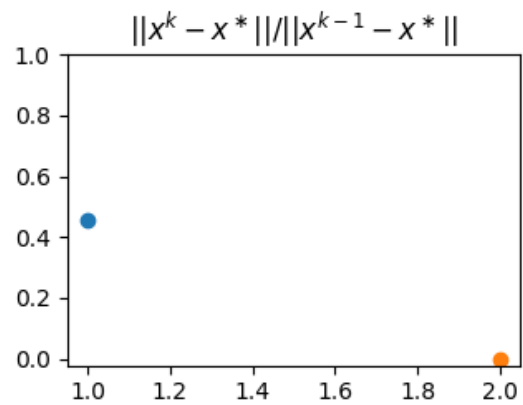
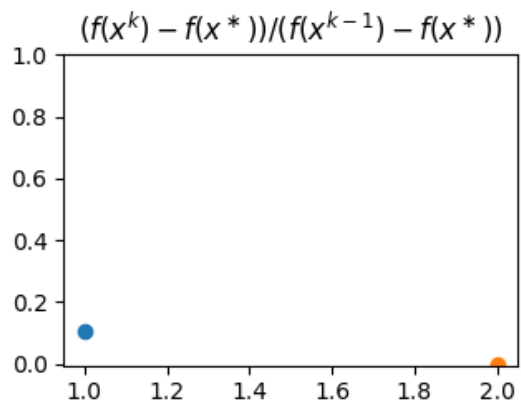
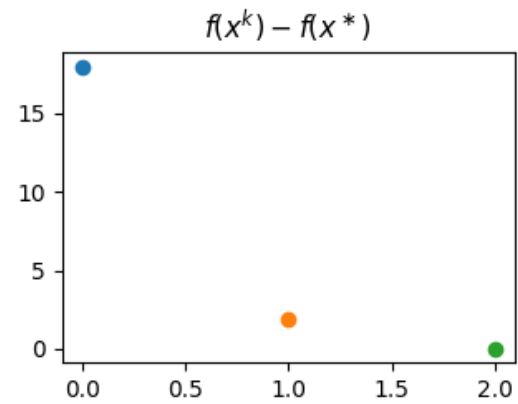
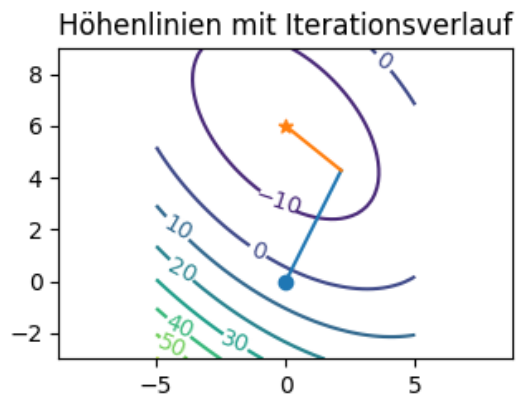
#PLOTS:

CG: A1



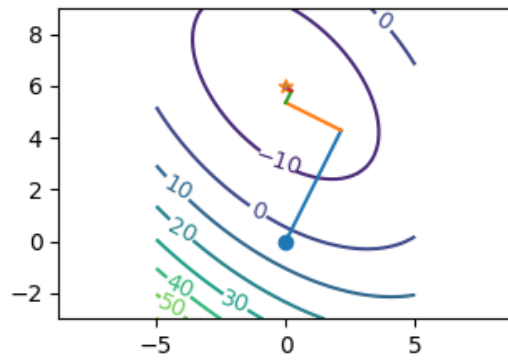
Gradientenabstieg: A1



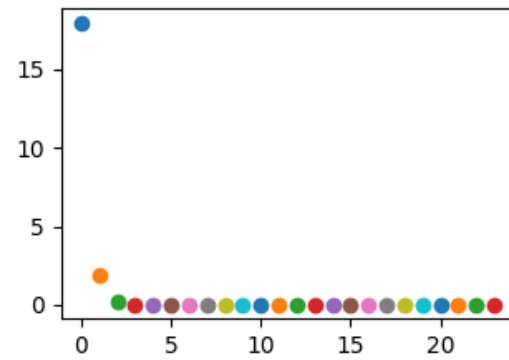


Gradientenabstieg: A2

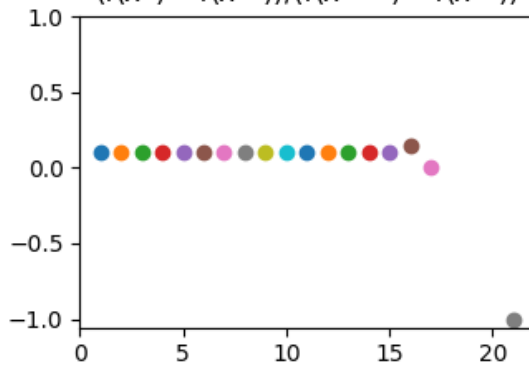
Höhenlinien mit Iterationsverlauf



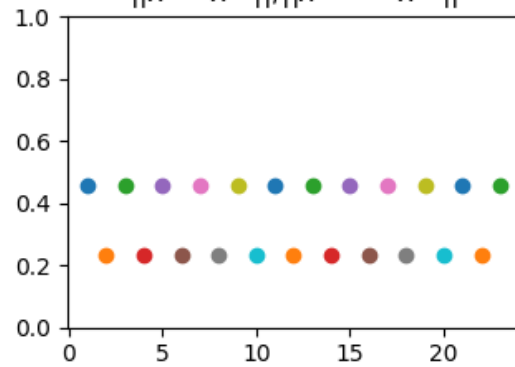
$f(x^k) - f(x^*)$



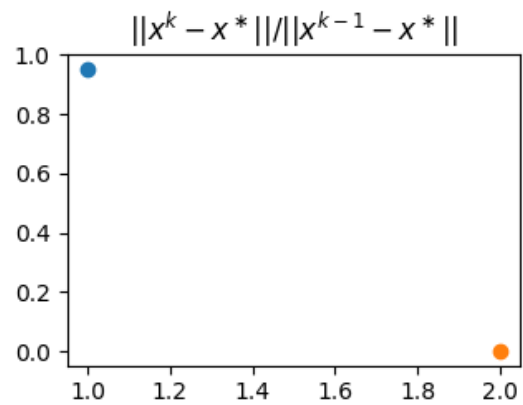
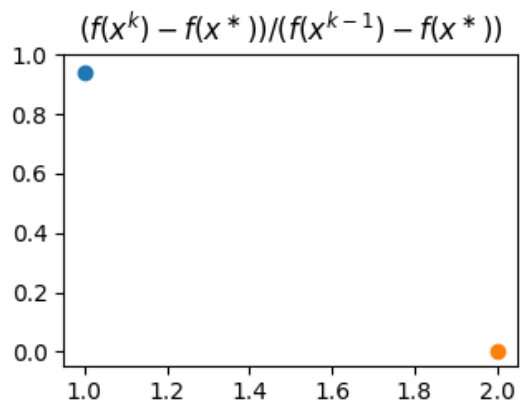
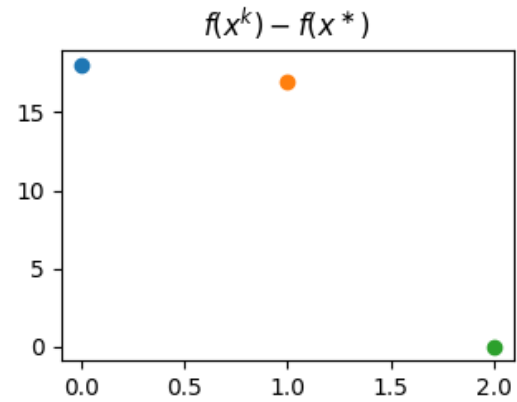
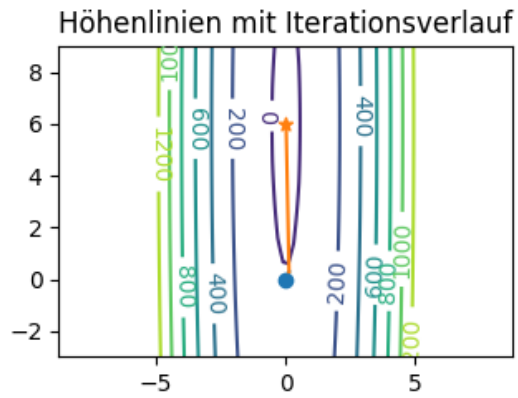
$(f(x^k) - f(x^*)) / (f(x^{k-1}) - f(x^*))$



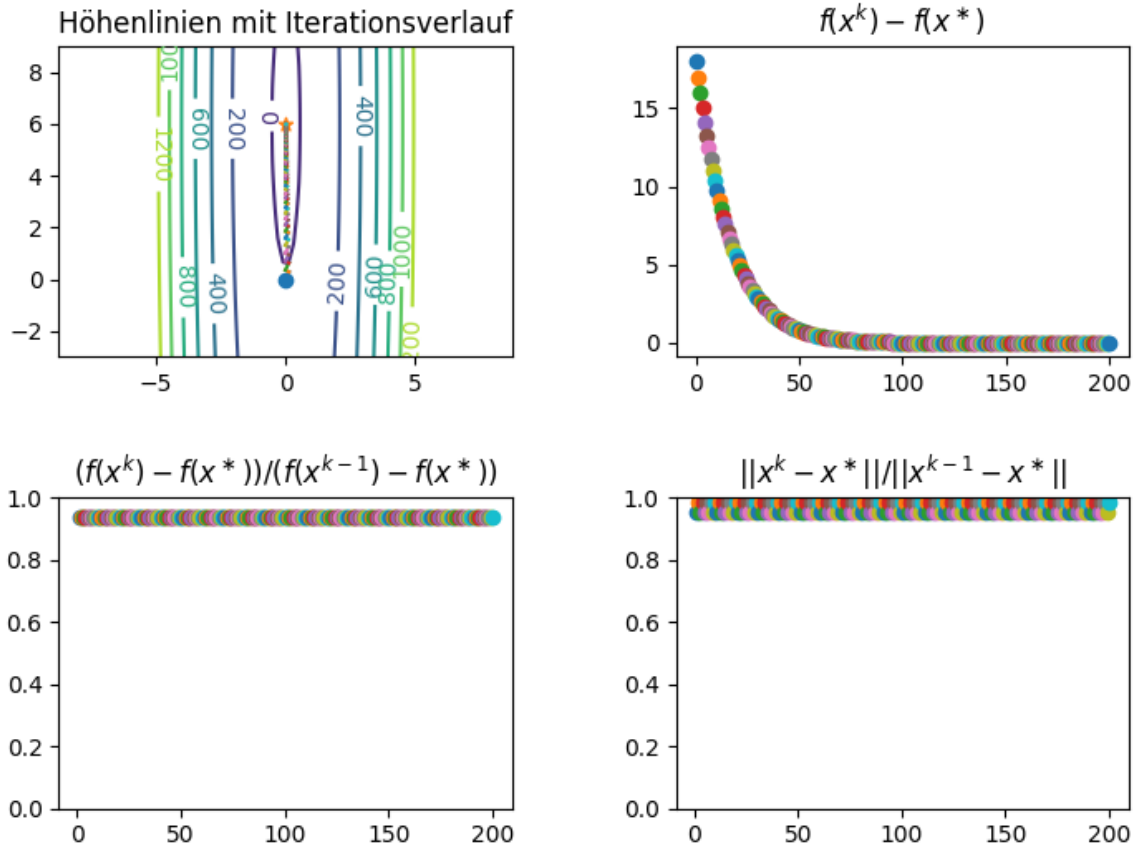
$\|x^k - x^*\| / \|x^{k-1} - x^*\|$



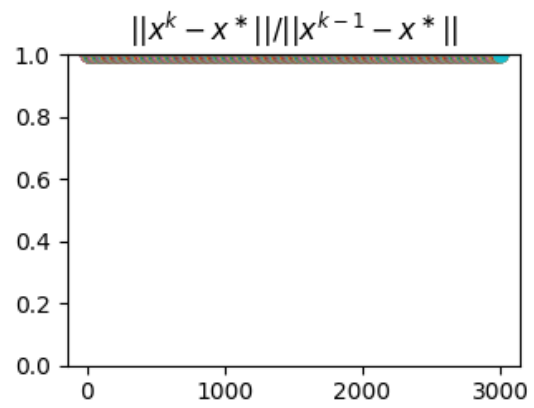
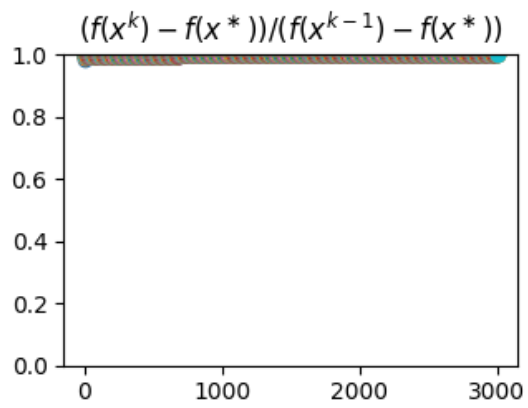
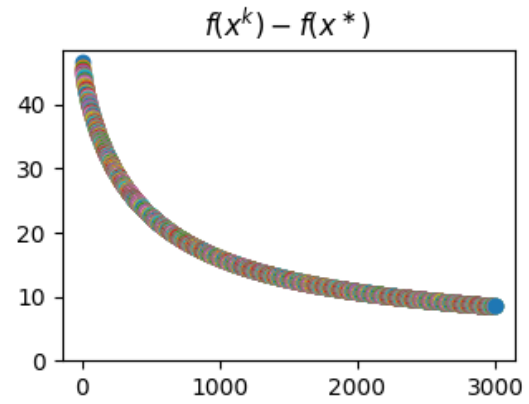
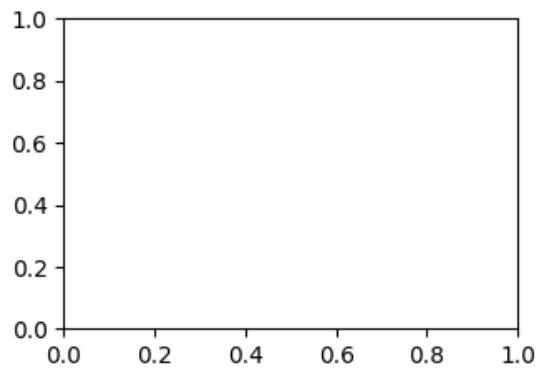
CG: A3



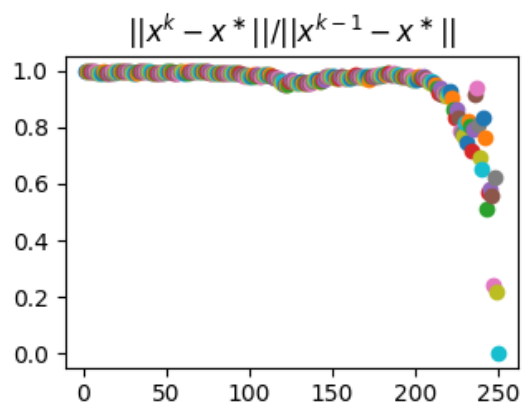
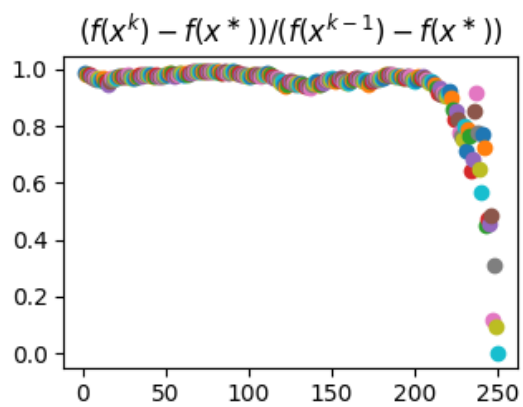
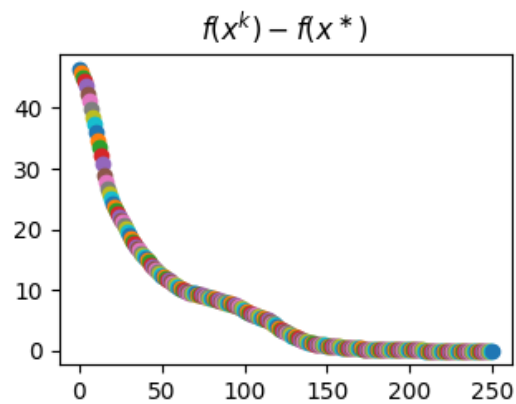
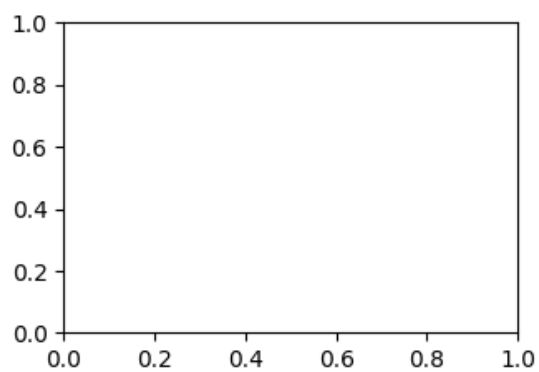
Gradientenabstieg: A3



Gradientenabstieg: A4



CG: A4



```
import numpy as np
import matplotlib.pyplot as plt

# Quadratische Funktion, Ã¼bernommen vom Blatt 5
def quadratic_function(x, A, b, c):
    """
    Quadratische Funktion

    Input:
        x: Variable,
        A: Matrix,
        b: Vektor,
        c: Skalar.

    Output:
        val: Skalar (f(x)),
        grad: Vektor ( \nabla f(x)),
        hess: Matrix ( \nabla^2 f(x)).
    """

    # Note: The ndarray x must have size (n,) for one coordinate reps. (m,n) for multiple coordinates
    val = 0.5 * np.einsum('...i,...i->...', x @ A, x) - np.inner(b, x) + c
    grad = x @ A - b # Note: x @ A instead of A @ x in order to deal with multiple coordinates
    hess = A
    return val, grad, hess

# Gradientenabstiegsverfahren, Ã¼bernommen vom Blatt 5, erweitert um Speichern der Anzahl der Iterationen.
def gradient_descent(f, x0, tol=1e-10, kmax=100, xstar=None):
    """
    Gradientenabstiegsverfahren

    Input:
        f: Funktion, wobei f(x) = (val, grad, hess),
        x0: Vektor (Startwert),
        tol: Skalar (Toleranz),
        kmax: Skalar (Maximale Anzahl an Iterationen),
        xstar: Vektor (Minimierer von f), optional.

    Output:
        log: Dictionary vom Verlauf der Optimierung.
    """
    # Dictionary zum Abspeichern der Ergebnisse.
    log = {
        'xstar': xstar, # TatsÃ¤chlicher Minimierer
        'val_star': None if xstar is None else f(xstar)[0], # TatsÃ¤chliches Minimum
        'x0': x0, # Startwert
        'xgd': None, # LÃ¶sung des Gradientenabstiegverfahrens
        'kgd': None, # Anzahl benÃ¶tigter Iterationen
        'x_list': [], # Liste der Iterierten
```

```

        'val_list': [],          # Liste des Funktionswerts in den Iterierten
        'norm_grad_list': [],   # Liste der Norm des Gradienten in den Iterierten
    }

```

```

k = 0
xk = x0
while True:
    valk, gradk, hessk = f(xk)

    log['x_list'].append(xk)
    log['val_list'].append(valk)
    log['norm_grad_list'].append(np.linalg.norm(gradk))

    if log['norm_grad_list'][-1] <= tol or k == kmax:
        break

    pk = -gradk
    alphak = - pk @ gradk / (pk @ hessk @ pk)
    xkplus1 = xk + alphak * pk

    k += 1
    xk = xkplus1

log['xgd'] = xk
log['kgd'] = k
return log

```

Plot-Funktion, Ã¼bernommen vom Blatt 5

```

def plot_iteration_process(log, title, f=None):
    """

```

*Funktion zur Erstellung von Plots zum Optimierungsverlauf anhand der Ergebnisse in log.
Die Funktion liefert fig zurÃ¼ck, Stellen Sie sie bitte dar,
indem Sie anschlieÃend plt.show() in Ihrer Hauptdatei aufrufen.*

Input:

```

    log: Dictionary mit mindestens folgenden EintrÃ¤gen
        log = {
            'xstar': Vektor,          # TatsÃ¤chlicher Minimierer (falls bekannt)
            'val_star': Skalar,       # TatsÃ¤chliches Minimum (falls bekannt)
            'x0': Vektor,            # Startwert
            'x_list': Liste,         # Liste der Iterierten
            'val_list': Liste,       # Liste des Funktionswerts in den Iterierten
            'norm_grad_list': Liste, # Liste der Norm des Gradienten in den Iterierten
        }

```

```

    title: Titel des Plots,

```

```

    f: Falls 2D: Optimierte Funktion, wobei  $f(x) = (val, grad, hess)$ , fÃ¼r Erstellung der HÃ¶henlinien

```

Output:

```

    fig: Erzeugte Figure mit den Plots
    """

```

In dieser Funktion initialisieren wir die Figure mit den Subplots

```

def initialize_figure():
    fig, axes = plt.subplots(nrows=2, ncols=2)
    fig.set_size_inches(8, 6)

```



```

fig.tight_layout(pad=4.0)
fig.suptitle(title)
return fig, axes

# Berechne Höhenlinien
def hoehen_linien():
    xx, yy, zz = None, None, None

    # Definitionsbereich wählen (x0 und xstar helfen abzuschätzen, wie groß dieser sein muss)
    n = 50
    xstar = log['xstar']
    x0 = log['x0']
    xaxis = np.linspace(min(x0[0], xstar[0]) - 5, max(x0[0], xstar[0]) + 5, n)
    yaxis = np.linspace(min(x0[1], xstar[1]) - 3, max(x0[1], xstar[1]) + 3, n)

    # Gitterpunkte und Funktionswerte auf den Gitterpunkten
    xx, yy = np.meshgrid(xaxis, yaxis, sparse=False, indexing='ij')
    zz, _, _ = f(np.concatenate((xx.reshape(n * n, 1), yy.reshape(n * n, 1)), axis=-1))
    zz = zz.reshape(n, n)
    return xx, yy, zz

# Initialisiere Abbildung
fig, ax = initialize_figure()
epsilon = np.finfo(float).eps # Vermeide Probleme mit Teilen durch 0 in den Plots später

# Plotten der Höhenlinien - Nur möglich für 2D-Funktionen (dann f als lambda-Funktion übergeben)
if f is not None:
    ax[0, 0].set_title('Höhenlinien mit Iterationsverlauf')
    ax[0, 0].set_aspect('equal', 'datalim') # Um die Orthogonalität der Suchrichtungen besser zu visualisieren
    xx, yy, zz = hoehen_linien()
    x0 = log['x0']
    xstar = log['xstar']
    x_list = log['x_list']
    CS = ax[0, 0].contour(xx, yy, zz)
    ax[0, 0].clabel(CS, inline=1, fontsize=10)
    ax[0, 0].scatter(*x0, marker='o')
    ax[0, 0].scatter(*xstar, marker='*')
    for k in range(len(x_list)-1):
        xk = x_list[k]
        xkplus1 = x_list[k+1]
        ax[0, 0].plot([xk[0], xkplus1[0]], [xk[1], xkplus1[1]])

# Plotten der Zielfunktionsfehler
ax[0, 1].set_title(r'$f(x^k) - f(x^{\ast})$')
val_list = log['val_list']
val_xstar = log['val_star'] if log['val_star'] is not None else log['val_list'][-1]
for k, valk in enumerate(val_list):
    ax[0, 1].scatter(k, valk - val_xstar)
ax[0, 1].set_ylim(min(0, ax[0, 1].get_ylim()[0]), max(1, ax[0, 1].get_ylim()[1]))

# Plotten der Konvergenzrate bezüglich der Zielfunktionswerte
ax[1, 0].set_title(r'$\frac{f(x^k) - f(x^{\ast})}{f(x^{k-1}) - f(x^{\ast})}$')
val_list = log['val_list']
val_xstar = log['val_star'] if log['val_star'] is not None else log['val_list'][-1]
for k in range(1, len(val_list)):
    valk = val_list[k]

```

```

    valkminus1 = val_list[k-1]
    if valkminus1 - val_xstar > epsilon:
        ax[1, 0].scatter(k, (valk - val_xstar) / (valkminus1 - val_xstar))
ax[1, 0].set_ylim(min(0, ax[1, 0].get_ylim()[0]), max(1, ax[1, 0].get_ylim()[1]))

# Plotten der Konvergenzrate bezüglich der Iterierten
ax[1, 1].set_title(r'$||x^k - x^{\ast}|| / ||x^{k-1} - x^{\ast}||$')
x_list = log['x_list']
xstar = log['xstar'] if log['xstar'] is not None else log['x_list'][-1]
for k in range(1, len(x_list)):
    xk = x_list[k]
    xkminus1 = x_list[k-1]
    if np.linalg.norm(xkminus1 - xstar) > epsilon:
        ax[1, 1].scatter(k, np.linalg.norm(xk - xstar) / np.linalg.norm(xkminus1 - xstar))
ax[1, 1].set_ylim(min(0, ax[1, 1].get_ylim()[0]), max(1, ax[1, 1].get_ylim()[1]))

return fig

```

#TERMINAL OUTPUT:

#PLOTS: